E-Munshi — Product Architecture & 30-Day Pilot Plan

Confidential Working Document · v1.0 · October 2026

> **Document Purpose**
> This document defines the product vision, system architecture, technology stack, security posture, cost model, and 30-day delivery roadmap for **E-Munshi** — an AI-powered accounting intelligence platform for Tally users. It is intended for technical review, client trust-building, and engineering kick-off.

Table of Contents

1. Product Vision & Positioning
2. Competitive Landscape & Feature Matrix
3. System Architecture Overview
4. Tally Integration Deep Dive
5. AI Orchestration & Financial Accuracy Engine
6. Technology Stack — Decisions with Rationale
7. Security Architecture & Data Handling
8. Database Design & Data Model
9. API Architecture
10. Frontend Architecture
11. Tally MCP Server — Adopted Repository Audit
12. Cost Model (₹ INR)
13. 30-Day Implementation Roadmap
14. Known Bottlenecks & Risk Register
15. Open Questions — Must Resolve Before Development
16. Pilot Release Checklist
17. Architecture Decision Records (ADRs)

---

1. Product Vision & Positioning

What is E-Munshi?

E-Munshi ("Munshi" — Hindi for trusted business bookkeeper/advisor) is an AI-powered financial intelligence platform designed for the 7+ million Indian businesses using Tally ERP. It connects to a user's existing Tally installation, extracts accounting data securely, and enables natural-language conversations with their own financial data — without replacing Tally, retraining accountants, or migrating books.

The Problem Being Solved

```
                        THE INFORMATION ACCESS GAP

Business Owner wants: "How much cash can I safely spend this month?"

TODAY (without E-Munshi)                    WITH E-Munshi
──────────────────────────────────────      ──────────────────────────
1. Open Tally Prime                         1. Open E-Munshi web app
2. Navigate to correct report               2. Type the question in chat
3. Set date range manually                  3. Receive answer with sources
4. Export to Excel                          ── 3 steps · under 5 seconds ──
5. Send to accountant on WhatsApp
6. Wait 2-3 days for reply
7. Get partial answer
8. Make decision with uncertainty
── 8 steps · 2-3 days · information loss ──
```

Target Users (Pilot Focus)

Persona	Pain Point	Value Delivered
Small Business Owner	Cannot read Tally; depends entirely on accountant for any number	Instant answers in plain English; cash position awareness 24/7
Chartered Accountant (CA)	Spends 60% of time on repetitive data extraction & reporting	Auto-generated MIS reports, receivables tracking, reconciliation drafts
Finance Manager / Controller	Manual month-end closing takes 5–7 days	Automated variance analysis, P&L with commentary, closing checklist
Business Partner / Investor	No visibility into business performance without asking CA	Read-only financial summaries, trend charts, and cash health indicators

---

2. Competitive Landscape & Feature Matrix

Competitor: Riko AI (`rikoai.in`)

Riko AI is the closest direct competitor — an Indian startup positioning itself as "The AI Operating Layer above Tally." Analysis of their live product reveals:

Feature	Riko AI	E-Munshi (Pilot v1)	E-Munshi (v2 Roadmap)
Natural Language Chat with Tally data	✅	✅	✅

Feature	Riko AI	E-Munshi (Pilot v1)	E-Munshi (v2 Roadmap)
P&L, Balance Sheet, Trial Balance	✅	✅	✅
Receivables / Payables Ageing	✅	✅	✅
Cash & Bank Position	✅	✅	✅
GST Reconciliation (2A/2B)	✅	Deferred	✅
Bank Reconciliation	✅	Deferred	✅
Monthly MIS Workpack (auto-generated)	✅	Deferred	✅
Write to Tally with Approval	✅ (paid tier)	Read-Only Pilot	✅
WhatsApp Integration	✅	Deferred	✅
Multi-company Support	✅	✅	✅
CA Firm / Multi-client Mode	✅ (Teams ₹20k/yr)	✅	✅
Indian-hosted Data	✅ Mumbai (AWS)	✅ Mumbai (AWS)	✅
Source-cited Answers	✅	✅	✅
Excel / PDF Export	✅	✅ Basic	✅ Full
Stock & Inventory Reports	✅	✅	✅
Audit-ready Answer Logs	✅	✅	✅
Mobile-first PWA	Android/iOS App	✅ PWA	✅

Pricing Comparison

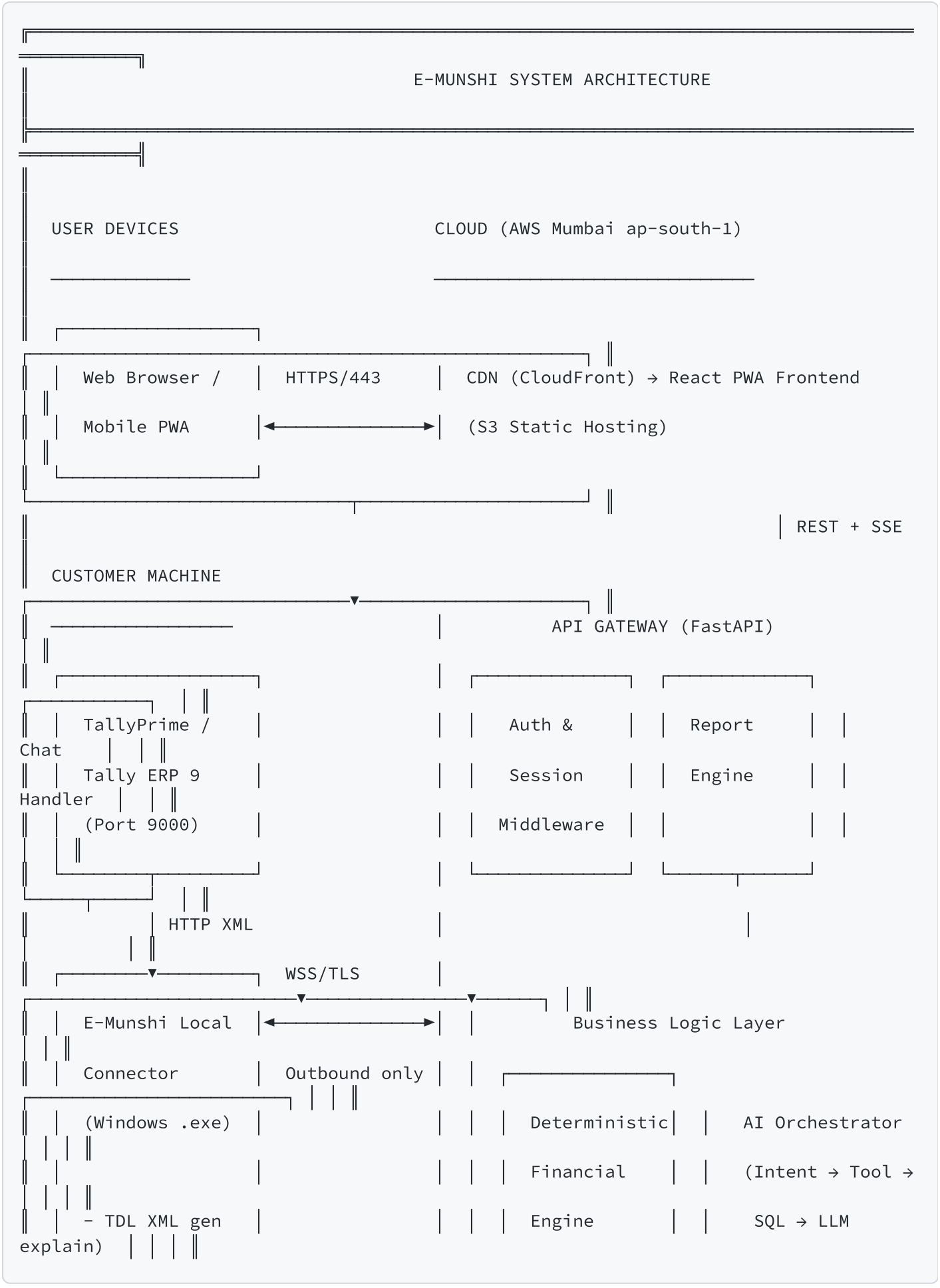
Plan	Riko AI	E-Munshi (Proposed)
Entry	₹5,000/year (1 user)	₹3,999/year or ₹399/month
Business	₹12,000/year (3 users)	₹9,999/year (5 users)
Teams / CA	₹20,000/year	₹18,000/year (10 users + white-label option)
Trial	7-day free, no card	14-day free, no card

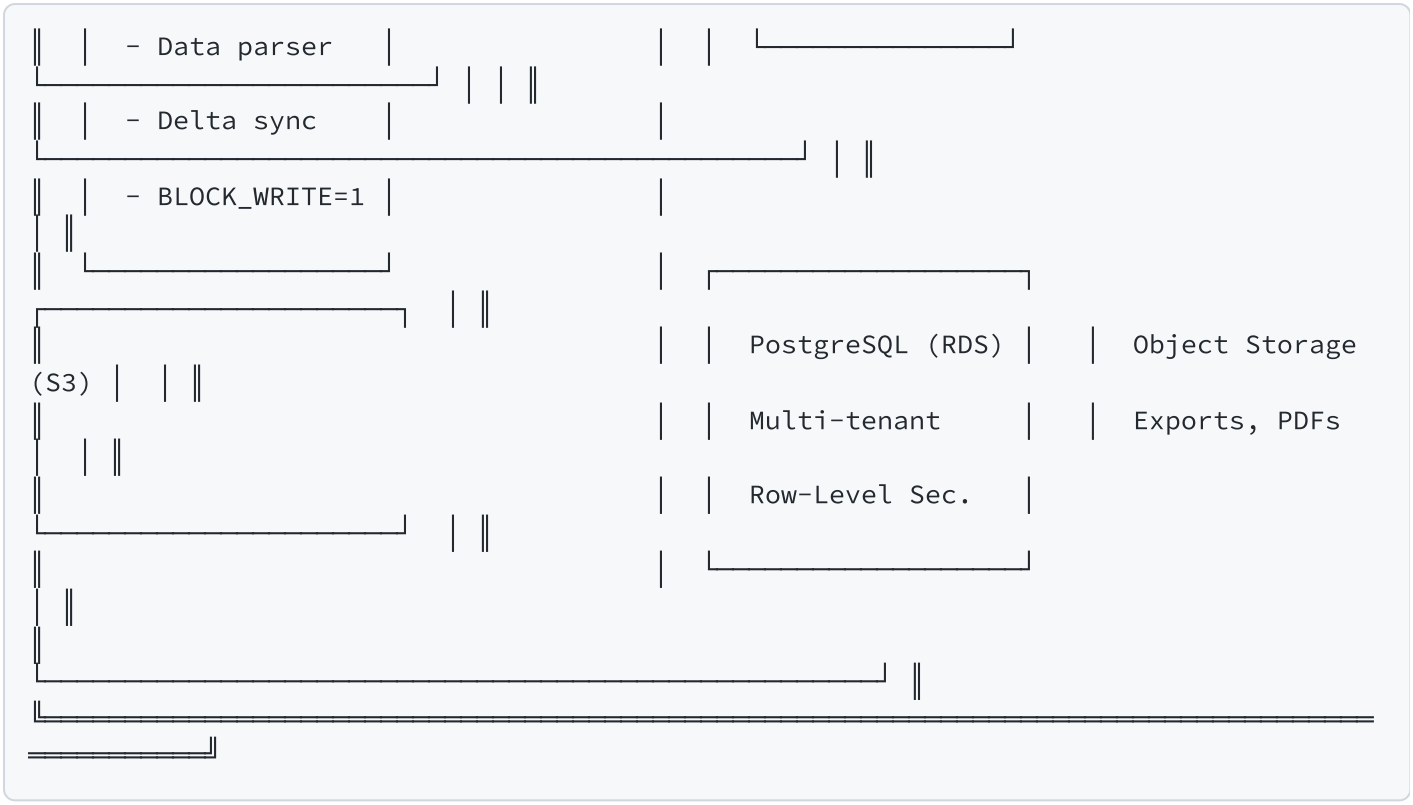
> **Strategic Differentiation**: E-Munshi enters at a 20% price discount with a longer trial and positions specifically for **CA firms and multi-client** use cases with better role-based access controls and a clear roadmap.

---

3. System Architecture Overview

High-Level Component Diagram





Trust Boundaries

```
TRUST ZONE 1: Customer Local Machine
  - TallyPrime process (fully customer-controlled)
  - E-Munshi Connector process (read-only, customer-installed)
  - All communication stays within this zone or goes outbound to E-Munshi cloud
  - No inbound connections accepted; no ports opened to the internet

TRUST ZONE 2: E-Munshi Cloud (AWS Mumbai)
  - All API traffic over HTTPS with TLS 1.2+
  - Tenant isolation enforced at database (RLS) and application layer
  - Secrets managed via AWS Secrets Manager / environment variables
  - No LLM provider receives raw financial data — only structured summaries

TRUST ZONE 3: External LLM Provider (Gemini / OpenAI)
  - Receives only: structured JSON tool results + user question text
  - Never receives: raw ledger data, PAN/GST numbers, party names in bulk
  - Provider data retention and processing terms reviewed before production
```

---

4. Tally Integration Deep Dive

> **This is the single most critical technical dependency.** Everything else in E-Munshi depends on this working reliably, at scale, and securely.

How Tally's XML Port Works

```
TALLY XML PORT PROTOCOL (Port 9000)

1. TallyPrime must be RUNNING and configured:
   F1 > Settings > Connectivity > Client/Server Configuration
   TallyPrime acts as = SERVER | Port = 9000

2. Request Format:
   POST http://localhost:9000/
   Content-Type: text/xml;charset=utf-16  ← CRITICAL: utf-16le NOT utf-8
   Body: TDL (Tally Definition Language) XML Envelope

3. Response Format:
   Content-Type: text/xml (utf-16le encoded)
   Body: Tally XML with requested data

4. KNOWN QUIRKS (already handled in tally-mcp-server):
   - Debits are negative numbers; Credits are positive
   - Dates formatted as "d-MMM-yyyy" (e.g., "1-Apr-2026")
   - Boolean fields return "Yes"/"No" strings, not true/false
   - Special chars like " in ledger names must be escaped with $$StrByCharCode:34

   - Large queries (50k+ vouchers) can block Tally UI for 10-30 seconds
```

Local Connector Architecture (Outbound-Only Agent)



Data Freshness Policy

Data Type	Freshness Source	Staleness Tolerance	User Indication
Cash & Bank Balance	Delta sync (15 min)	30 minutes	"As of HH:MM today"

Data Type	Freshness Source	Staleness Tolerance	User Indication
Receivables / Outstanding	Delta sync (15 min)	2 hours	"As of HH:MM today"
Trial Balance / P&L	On-demand from Tally	Must be live	"Fetched live from Tally"
Ledger Statement	On-demand from Tally	Must be live	"Fetched live from Tally"
Stock Summary	Delta sync (30 min)	4 hours	"As of HH:MM today"
Master Data (Ledger Names)	On-demand refresh	24 hours	Background refresh

> **If Tally is offline when a user queries**: E-Munshi serves the last synced data with a visible freshness warning badge (e.g., `⚠ Data from 3 hours ago — Tally connector is offline`). It **never** serves stale data silently.

Supported Tally Versions

Version	Status	Notes
TallyPrime 3.x / 4.x / 5.x	✅ Fully Supported	Primary target
TallyPrime 2.x	✅ Supported	XML port stable
Tally.ERP 9 (Release 6+)	✅ Supported	Widely deployed
Tally.ERP 9 (< Release 6)	⚠ Limited	Missing some bill-wise fields
Tally Educational Edition	❌ Not Supported	Date range restrictions cause invalid data
Tally Silver (Single User)	✅ Supported	One Tally instance per connector
Tally Gold (Multi-User)	✅ Supported	Multiple concurrent sessions
Tally on cloud / TSS	⚠ Test Required	Port 9000 accessibility must be verified

---

5. AI Orchestration & Financial Accuracy Engine

> **Core Design Principle: The LLM is an explainer, not a calculator.**
> Every financial figure displayed to the user comes from deterministic code execution against verified Tally data. The LLM only translates user intent into structured tool calls, and then formats deterministic results into readable language.

Query Processing Flow

```
USER: "Who owes me the most money and for how long?"
                │
                ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│  STEP 1: INTENT CLASSIFICATION & ENTITY EXTRACTION                           │
│  ──────────────────────────────────────────────────                          │
│  Input: user question + conversation context + current company/period        │
│  LLM Task: structured JSON output (NOT financial calculation)                │
│                                                                              │
│  Output: {                                                                   │
│    "intent": "QUERY_RECEIVABLES_AGING",                                      │
│    "company": "Acme Corp Pvt Ltd",   ← from session context                  │
│    "as_of_date": "2026-10-23",       ← today (resolved deterministically)    │
│    "sort_by": "outstanding_amount",                                          │
│    "order": "DESC",                                                          │
│    "limit": 10                                                               │
│  }                                                                           │
└──────────────────────────────────────────────────────────────────────────────┘
                                    │
                ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│  STEP 2: PARAMETER GUARD & PERMISSION CHECK                                  │
│  ──────────────────────────────────────────                                  │
│  - Verify user has access to requested company (server-side, from DB)        │
│  - Validate date range (not future, within available data)                   │
│  - Validate intent is in ALLOWLIST (no arbitrary SQL, no unrestricted tools) │
│  - Apply tenant_id scope to ALL database operations                          │
└──────────────────────────────────────────────────────────────────────────────┘
                                    │
                ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│  STEP 3: DETERMINISTIC DATA RETRIEVAL                                        │
│  ────────────────────────────────────                                        │
│  SELECT                                                                      │
│    party_name,                                                               │
│    SUM(outstanding_amount) AS total_outstanding,                             │
│    MAX(overdue_days) AS max_overdue_days,                                    │
│    COUNT(bill_reference) AS bill_count                                       │
│  FROM bill_allocations                                                       │
│  WHERE tenant_id = :tenant_id                                                │
│    AND company_id = :company_id                    ← HARDCODED from session  │
│    AND nature = 'receivable'                                                 │
│    AND as_of_date <= :as_of_date                                             │
│    AND outstanding_amount > 0                                                │
│  GROUP BY party_name                                                         │
│  ORDER BY total_outstanding DESC                                             │
│  LIMIT 10                                                                    │
└──────────────────────────────────────────────────────────────────────────────┘
                                    │
                ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│  STEP 4: DETERMINISTIC FINANCIAL COMPUTATION                                 │
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│  ──────────────────────────────────────────                              │
│  Python (no LLM involved):                                               │
│  - Calculate aging buckets: 0-30, 31-60, 61-90, 90+ days                 │
│  - All arithmetic uses Python Decimal(2) — no float point errors         │
│  - Total = sum(row.outstanding) verified against trial balance           │
│  - Source metadata: company, period, data_freshness, row_count           │
│  - Result includes: source_voucher_ids, report_timestamp                 │
└──────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌──────────────────────────────────────────────────────────────────────────┐
│  STEP 5: GROUNDED LLM RESPONSE GENERATION                                │
│  ──────────────────────────────────────                                  │
│  System prompt includes:                                                 │
│  - "You are a financial assistant. Do NOT alter any numbers in the data."│
│  - "Do NOT perform arithmetic. Use only the pre-calculated figures."     │
│  - "If data is stale, mention the as-of date prominently."               │
│  - Structured result JSON (amounts are strings, not floats, to prevent LLM│
│    rounding/reformatting)                                                │
│                                                                          │
│  Output: Natural language + table + source citation + suggested follow-ups│
└──────────────────────────────────────────────────────────────────────────┘
```

Pilot Allowlisted Tools (v1 Scope)

Tool	Inputs	Data Source	Safe
`get_cash_bank_balance`	company, as_of_date	Cached sync	✅
`get_profit_loss`	company, from_date, to_date	Tally live / cache	✅
`get_trial_balance`	company, from_date, to_date	Tally live	✅
`get_outstanding_bills`	company, nature, as_of_date	Cached sync	✅
`get_receivables_aging`	company, as_of_date, buckets	Deterministic computation	✅
`get_ledger_statement`	company, ledger_id, from_date, to_date	Tally live	✅
`get_stock_summary`	company, from_date, to_date	Cached sync	✅
`get_top_customers`	company, from_date, to_date, by_metric	Deterministic computation	✅
`get_expense_analysis`	company, from_date, to_date, group	Deterministic computation	✅

Tool	Inputs	Data Source	Safe
`compare_periods`	company, period1, period2, metric	Deterministic computation	✅

Anti-Hallucination Safeguards

1. **Prompt injection defense**: Ledger names, narrations, and party names from Tally are treated as untrusted input and never injected directly into LLM prompts unescaped.
2. **Number type enforcement**: All amounts are passed to the LLM as JSON strings (e.g., `"1245000"` not `1245000.0`) to prevent LLM reformatting or rounding.
3. **Source citation mandatory**: Every response must include `source_metadata` — if the LLM generates a figure not present in the structured data, the backend validation layer rejects it.
4. **Fallback on insufficient data**: If data is unavailable or Tally connector is offline, the system returns a structured error — the LLM never fabricates a figure.

---

6. Technology Stack — Decisions with Rationale

Selected Stack

E-MUNSHI TECHNOLOGY STACK

Layer	Technology	Rationale
Frontend	React 19 + Vite TypeScript Vanilla CSS / CSS Modules Recharts / Chart.js TanStack Query (data fetch)	Fastest build tooling; large ecosystem; Vite HMR for rapid UI iteration
Backend API	Python 3.12 + FastAPI Pydantic v2 (validation) SQLAlchemy 2.x (async ORM) Alembic (migrations)	Best-in-class AI/data ecosystem (LangChain, pandas, decimal); async-native; auto-docs
AI Layer	Google Gemini 2.0 Flash (primary model) LiteLLM (provider wrapper)	Chosen for: best structured JSON output, lowest cost per token (~₹0.008/query), Gemini's strong Indian lang understanding, <800ms P50 latency. LiteLLM allows provider swap without code rewrite.
Database	PostgreSQL 16 (AWS RDS Mumbai / Supabase ap-south-1) Row-Level Security (RLS)	ACID guarantees for accounting data; Row-Level Security for tenant isolation; native JSONB
Caching	Redis 7 (Upstash / ElastiCache)	Session tokens, rate limiting, sync job queuing
Auth	Supabase Auth (JWT + refresh tokens)	Email+password + Google OAuth; India-hosted; free tier generous
Connector (Local Agent)	Node.js (TypeScript) Packaged via pkg / SEA Single .exe (no deps)	Re-uses tally-mcp-server TDL XML templates (ISC license — commercially free)
File Storage	AWS S3 (Mumbai) Pre-signed URLs (15 min TTL)	Signed URLs for exports; Indian data residency
CI/CD	GitHub Actions	Free for private repos; integrates with AWS deploy
Hosting	AWS Lightsail / EC2 (Mumbai) + CloudFront CDN	Predictable cost; Mumbai region; easy scaling

Monitoring	Sentry (errors) Grafana Cloud (metrics)	Both have generous free tiers for pilot phase

LLM Provider Decision: Google Gemini 2.0 Flash

Rationale for Gemini over OpenAI GPT-4o-mini:

Criterion	Gemini 2.0 Flash	GPT-4o-mini
Cost per 1M input tokens	~$0.075	~$0.15
Cost per 1M output tokens	~$0.30	~$0.60
Structured JSON output	✅ Native	✅ Native
Multilingual (Hindi, Gujarati)	✅ Excellent	✅ Good
Context window	1M tokens	128K tokens
Google India data processing	Partially available	Requires Azure OpenAI
Estimated cost per query	₹0.008	₹0.015

LiteLLM wrapper ensures if provider needs to change (pricing, outage, compliance), only 2 lines of config change.

AI Development Tooling

> **Note**: Given the team is primarily 1 full-stack developer, AI coding tools are explicitly part of the development workflow.

Tool	Purpose	Plan
Claude Code (Anthropic)	Primary AI coding assistant for complex backend logic	Pro subscription — highest ROI for financial logic
GitHub Copilot	In-editor autocomplete for boilerplate and tests	Individual subscription ₹800/month
Cursor	AI-first code editor for rapid UI scaffolding	Hobby plan — free tier sufficient for pilot

---

7. Security Architecture & Data Handling

Security Layers

```
LAYER 1: TRANSPORT SECURITY
  ✅ TLS 1.2+ enforced on all endpoints (HSTS headers)
  ✅ Connector WebSocket uses WSS (TLS) — never plain WS
  ✅ HTTPS-only cookies with Secure + HttpOnly + SameSite=Strict flags
  ✅ Certificate pinning on connector (prevents MITM on local machine)

LAYER 2: AUTHENTICATION & SESSION
  ✅ JWT access tokens (15-minute expiry)
  ✅ Refresh tokens (7-day, rotated on each use)
  ✅ Google OAuth 2.0 PKCE flow
  ✅ Connector pairing uses one-time token (expires in 10 minutes)
  ✅ All token validation server-side; no trust in client-asserted IDs

LAYER 3: AUTHORIZATION (EVERY API CALL)
  ✅ Role-based: OWNER / ACCOUNTANT / VIEWER
  ✅ Tenant isolation: server extracts org_id from JWT — client CANNOT inject it
  ✅ Company-level access: user must be explicitly granted access to each company
  ✅ PostgreSQL Row-Level Security (RLS) as defense-in-depth

LAYER 4: INPUT VALIDATION
  ✅ Pydantic v2 schema validation on all API inputs
  ✅ Date range validation (no future dates, no 100-year spans)
  ✅ Ledger/company names validated against authorized list (no SQL injection via
names)
  ✅ Query intent validated against ALLOWLIST before any Tally or DB query

LAYER 5: DATA PROTECTION
  ✅ AES-256 encryption at rest (AWS RDS encryption)
  ✅ All Tally data tagged with tenant_id + company_id in every table
  ✅ Exports stored in private S3 with short-lived (15 min) pre-signed URLs
  ✅ No raw financial data in LLM prompts — structured summaries only
  ✅ PAN, GST numbers, bank account numbers masked in logs

LAYER 6: RATE LIMITING & ABUSE PREVENTION
  ✅ Per-user: 50 chat queries/hour
  ✅ Per-organization: 500 API calls/hour
  ✅ Connector sync rate: max 1 sync per 5 minutes per company
  ✅ Redis-backed rate limiter (sliding window algorithm)

LAYER 7: AUDIT LOGGING
  ✅ Every chat query logged: user, org, company, question, tool_used, timestamp
  ✅ Every report fetch logged: user, company, report_type, period, source
  ✅ Every sync run logged: connector_id, company, record_count, watermark, status
  ✅ Logs stored 90 days; queryable by organization admin
```

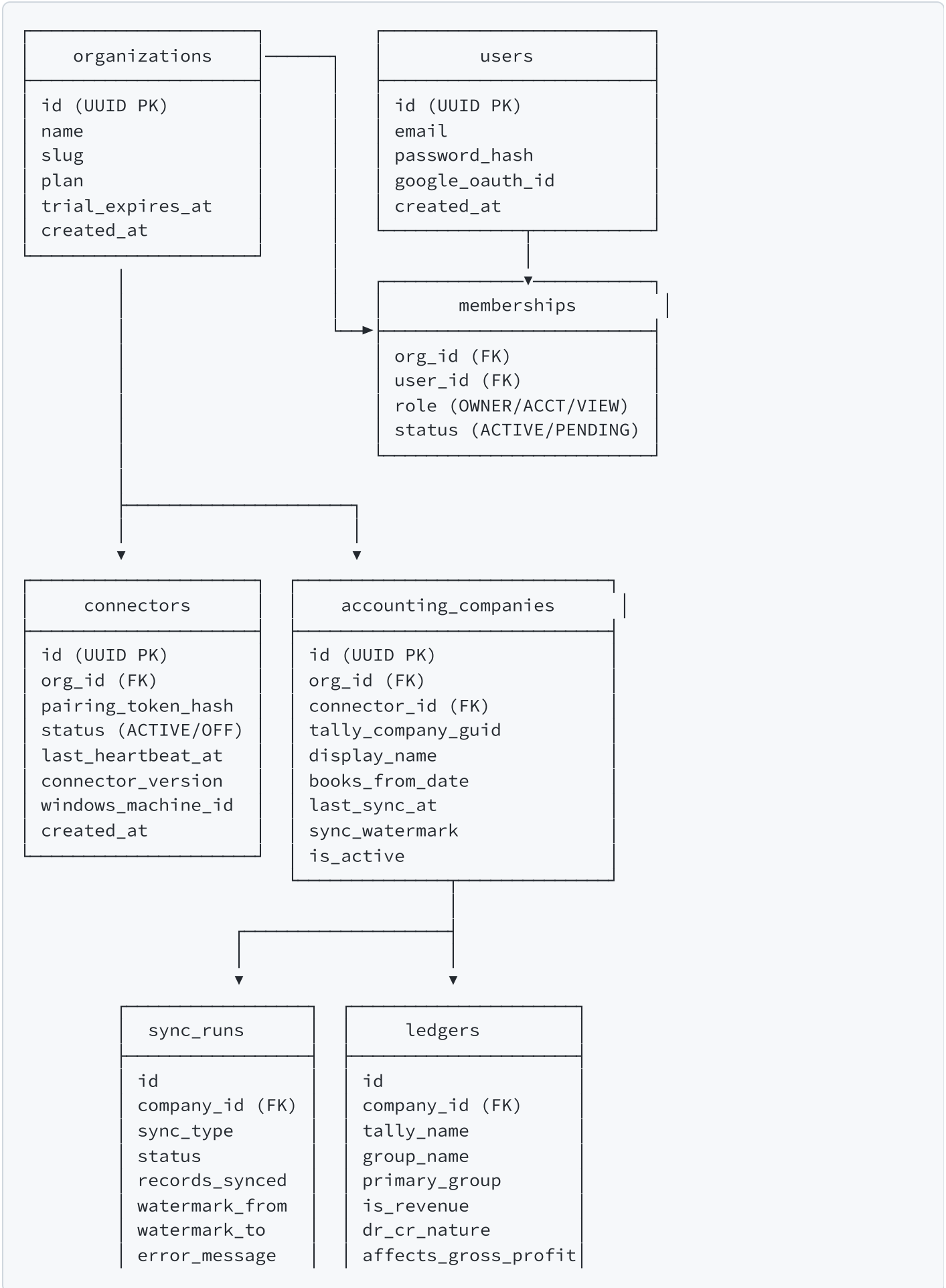
Sensitive Data Handling

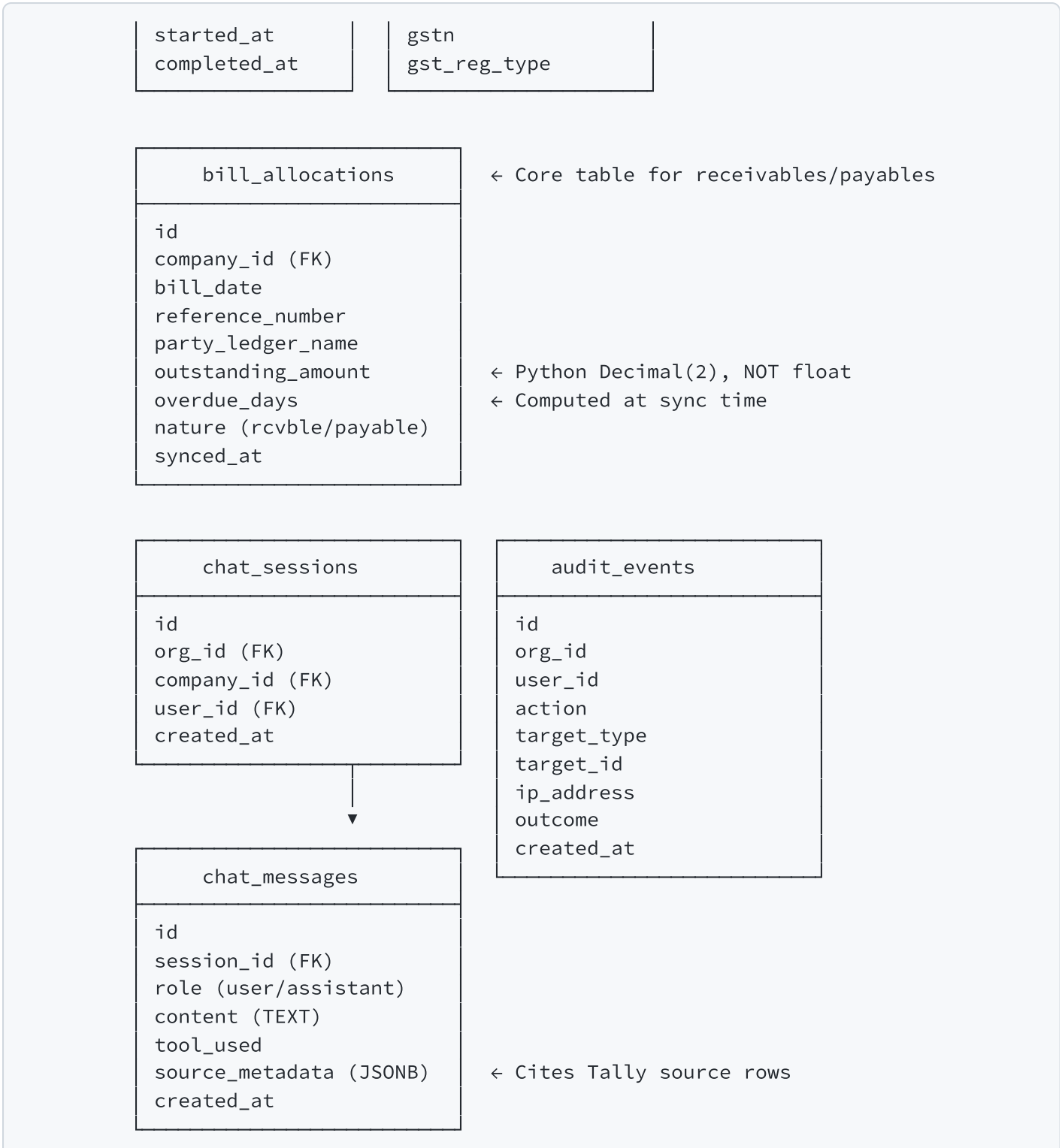
Data Type	Stored In	Encrypted	Transmitted to LLM	Retention
User email / password hash	PostgreSQL	✅ bcrypt	❌ Never	Account lifetime
Tally voucher data	PostgreSQL	✅ at rest	❌ Never raw	24 months
Financial totals & balances	PostgreSQL	✅ at rest	✅ Structured JSON only	24 months
Chat history	PostgreSQL	✅ at rest	✅ Yes (user questions)	12 months
GST / PAN numbers	PostgreSQL	✅ at rest	❌ Masked in logs	Account lifetime
JWT tokens	Redis	✅ in-transit	❌ Never	15 minutes
Connector pairing token	Local keystore	✅ DPAPI	❌ Never	Permanent until revoked
Export files (XLSX/PDF)	S3 (private)	✅ at rest	❌ Never	7 days TTL

---

8. Database Design & Data Model

Entity Relationship Overview




started_at
completed_at
gstn
gst_reg_type
bill_allocations
← Core table for receivables/payables
id
company_id (FK)
bill_date
reference_number
party_ledger_name
outstanding_amount
← Python Decimal(2), NOT float
overdue_days
← Computed at sync time
nature (rcvble/payable)
synced_at
chat_sessions
id
org_id (FK)
company_id (FK)
user_id (FK)
created_at
audit_events
id
org_id
user_id
action
target_type
target_id
ip_address
outcome
created_at
chat_messages
id
session_id (FK)
role (user/assistant)
content (TEXT)
tool_used
source_metadata (JSONB)
← Cites Tally source rows
created_at

Row-Level Security Policy (PostgreSQL RLS)

```
-- Every table with financial data has this policy:
ALTER TABLE bill_allocations ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON bill_allocations
  USING (
    company_id IN (
      SELECT ac.id FROM accounting_companies ac
      JOIN memberships m ON m.org_id = ac.org_id
      WHERE m.user_id = current_setting('app.current_user_id')::uuid
        AND m.status = 'ACTIVE'
    )
  );
```

---

9. API Architecture

Versioned API Structure

```
Base URL: https://api.emunshi.in/v1/

POST   /auth/login                      → Login with email+password
POST   /auth/google                     → Google OAuth callback
POST   /auth/refresh                    → Refresh JWT
DELETE /auth/logout                     → Invalidate tokens

GET    /me                              → Current user + org memberships
POST   /organizations                   → Create organization
GET    /organizations/:id               → Get org details

GET    /companies                       → List authorized Tally companies
POST   /connectors/pair-init            → Generate pairing token (10-min TTL)
GET    /connectors/status               → Connector health + last heartbeat

POST   /sync/delta                      → Connector POST (auth by pairing token)
GET    /sync/runs                       → List recent sync runs with status

GET    /reports/trial-balance           → Verified trial balance
GET    /reports/profit-loss             → P&L for period
GET    /reports/balance-sheet           → Balance sheet as of date
GET    /reports/outstanding-bills       → Receivable / Payable outstanding
GET    /reports/ledger-statement        → Ledger account statement
GET    /reports/stock-summary           → Stock summary

POST   /chat/sessions                   → Start new chat session
POST   /chat/sessions/:id/messages      → Send query; returns SSE stream
GET    /chat/sessions                   → Chat history list

GET    /files/:file_id                  → Download export (pre-signed redirect)

GET    /audit/events                    → Audit log (admin/owner only)

# All endpoints return:
# { "data": {...}, "meta": { "request_id": "...", "timestamp": "..." } }
# Errors return:
# { "error": { "code": "UNAUTHORIZED", "message": "...", "request_id": "..." } }
```

Chat Streaming (Server-Sent Events)

```
POST /chat/sessions/:id/messages
Body: { "message": "Who owes me the most?" }

Response: text/event-stream

event: thinking
data: {"status": "Fetching receivables data..."}

event: thinking
data: {"status": "Computing aging buckets..."}

event: result
data: {
  "answer": "Your top debtor is Sunrise Traders...",
  "table": [...],
  "source": { "report": "bills_outstanding", "as_of": "2026-10-23",
"freshness_minutes": 12 },
  "suggestions": ["Show all overdue bills", "Send payment reminder draft"]
}

event: done
data: {}
```

---

10. Frontend Architecture

Application Structure

```
src/
├── App.tsx                    ← Route definitions
├── main.tsx                   ← Entry point
│
├── pages/
│   ├── auth/
│   │   ├── LoginPage.tsx      ← Email/password + Google OAuth
│   │   └── OnboardingPage.tsx ← Org creation + first connector pairing
│   ├── dashboard/
│   │   ├── DashboardPage.tsx  ← Main financial overview
│   │   └── widgets/           ← Cash, Receivables, P&L cards
│   ├── chat/
│   │   ├── ChatPage.tsx       ← Conversational AI interface
│   │   └── MessageBubble.tsx  ← Source-cited financial response
│   ├── reports/
│   │   ├── TrialBalancePage.tsx
│   │   ├── OutstandingBillsPage.tsx
│   │   └── LedgerStatementPage.tsx
│   └── settings/
│       ├── ConnectorPage.tsx  ← Download connector + pairing flow
│       └── TeamPage.tsx       ← User management + roles
│
├── components/
│   ├── CompanySelector.tsx    ← Dropdown with sync status badge
│   ├── SyncStatusBadge.tsx    ← Green/Yellow/Red indicator
│   ├── SourceCitation.tsx     ← "Source: Tally, as of 14:23 today"
│   ├── DataFreshnessWarn.tsx ← Warning when data is stale
│   └── FinancialTable.tsx     ← Sortable, exportable data table
│
├── hooks/
│   ├── useAuth.ts             ← Auth state + token management
│   ├── useCompany.ts          ← Selected company context
│   └── useChat.ts             ← SSE streaming chat hook
│
└── services/
    ├── api.ts                 ← Axios instance with JWT interceptor
    └── queryClient.ts         ← TanStack Query config
```

Dashboard Design (Key Screens)

Screen 1 — Dashboard

- Top: Company selector dropdown + Connector sync status badge (●LIVE / ●SYNCING / ●OFFLINE)
- Row 1: Cash & Bank (₹X.XL) card | Sales This Month (₹X.XL) vs Last Month (▲/▼%) card
- Row 2: Top 5 Debtors bar chart | Overdue Bills summary card
- Row 3: Quick question chips: "Show P&L", "Who owes me money?", "Cash position"

Screen 2 — AI Chat

- Full-page chat interface, dark-accented sidebar with history

- Message bubbles with source citation footer on each answer
- Animated streaming indicator while query is processing
- Suggested follow-up questions after each answer

---

11. Tally MCP Server — Adopted Repository Audit

The repository `dhananjay1405/tally-mcp-server` was cloned and audited on **October 23, 2026**.

Adoption Decision Summary

Component	Decision	Reason
TDL XML templates (`templates/`)	✅ **ADOPT**	CA-verified TDL generation; solves hardest Tally quirks
Field definitions (`definition.mts`)	✅ **ADOPT**	100+ Tally fields mapped with correct data types and sign conventions
Tally HTTP client (`tally.mts`)	✅ **ADOPT**	Correct utf-16le handling, double-quote escaping, XML parsing
Financial validation logic	✅ **ADOPT**	Dr/Cr balancing check using integer arithmetic (avoids JS float errors)
In-memory PGlite database	❌ **DISCARD**	Ephemeral, single-process, not suitable for cloud multi-tenant use
Express OAuth server (`server.mts`)	❌ **DISCARD**	Stores tokens in-memory (lost on restart); no tenant concept
Direct live Tally querying	❌ **REPLACE**	Replace with delta-sync architecture; live queries block Tally UI
Write tools (voucher-journal, ledger-create-update, delete-master)	❌ **DISABLED**	`BLOCK_WRITE=1` enforced; zero write operations in pilot

---

12. Cost Model (₹ INR)

30-Day Pilot Infrastructure (Recommended Configuration)

Service	Provider	Spec	Monthly Cost	Notes
API Server	AWS Lightsail	2 vCPU, 4GB RAM (Mumbai)	₹1,650	Includes 80GB SSD + 4TB transfer
Database	Supabase Pro	PostgreSQL (ap-south-1)	₹2,100	Row-Level Security, backups

Service	Provider	Spec	Monthly Cost	Notes
Redis	Upstash	Serverless Redis	₹0–₹400	Free up to 10k commands/day
Frontend CDN	AWS CloudFront + S3	Static + CDN	₹200	Negligible for pilot traffic
LLM (Gemini Flash)	Google Cloud	~15,000 queries/month	₹1,200	@ ₹0.008/query avg
Object Storage	AWS S3 (Mumbai)	Exports + logs	₹150	First 5GB free
Email (Transactional)	Resend / Brevo	Up to 3,000/month	₹0	Free tier sufficient
Monitoring	Sentry + Grafana Cloud	Error tracking	₹0	Free tiers
Domain (.in + SSL)	Namecheap / GoDaddy	emunshi.in + cert	₹800/year	~₹67/month
TOTAL MONTHLY			**~₹5,770/month**	

Engineering Tooling (Monthly)

Tool	Cost	Purpose
Claude Code (Anthropic Pro)	~₹1,700/month	Primary AI coding assistant
GitHub Copilot	₹800/month	In-editor autocomplete
GitHub (private repos)	₹0	Free for individual

One-Time Setup Costs

Item	Low	Recommended	Notes
Domain registration (emunshi.in / .com)	₹800	₹2,500	.in cheaper; .com preferred for brand
Windows Code Signing Cert (connector .exe)	₹0 (test mode)	₹18,000/yr	Without cert: "Unknown Publisher" warning
AWS account setup + initial credits	₹0	₹0	AWS Free Tier covers initial 12 months

3-Month Total Budget Projection

Item	3-Month Cost
Infrastructure (cloud)	₹17,310
LLM API (scaled to 5k users → 75k queries)	₹600
Domain + Email	₹2,500
AI development tooling	₹7,500
Code signing cert (optional)	₹0 (pilot skip)
TOTAL	**₹27,910**

> **Budget Verdict**: The full pilot can be run for **under ₹30,000** for 3 months — well within the ₹50,000–₹1,00,000 budget, leaving ₹20,000–₹70,000 for onboarding, marketing, and security testing.

---

13. 30-Day Implementation Roadmap

Weekly Sprint Plan

```
═══════════════════════════════════════════════════════════════════════════
 WEEK 1 (Days 1–7): Foundation, Proof-of-Concept, Database
═══════════════════════════════════════════════════════════════════════════

 Day 1–2: Environment & Architecture Setup
  ☐ Register emunshi.in domain
  ☐ Set up GitHub private repo (monorepo: packages/connector, apps/api, apps/web)
  ☐ Initialize FastAPI backend project with Pydantic, SQLAlchemy, Alembic
  ☐ Initialize React + Vite frontend project
  ☐ Set up PostgreSQL on Supabase (Mumbai region)
  ☐ Write first Alembic migration: organizations, users, memberships
  ☐ Configure GitHub Actions CI (lint + type-check on push)

 Day 3–4: Tally Connector Core (Based on tally-mcp-server)
  ☐ Extract tally-mcp-server TDL templates and definition into packages/connector
  ☐ Implement delta sync logic (ALTEREDID watermark tracking)
  ☐ Test against a live TallyPrime instance on local Windows machine
  ☐ Verify: company list, chart of accounts, vouchers, bill-wise outstanding
  ☐ Compare extracted trial balance against Tally's own P&L report
  EXIT CRITERION: Can extract and verify 5 report types from real Tally data

 Day 5–6: Cloud Ingestion API + Auth
  ☐ Build POST /sync/delta endpoint (connector auth via pairing token)
  ☐ Build POST /auth/login and POST /auth/google (Supabase Auth)
  ☐ Write database migrations for: connectors, accounting_companies, sync_runs
  ☐ Implement connector pairing flow (one-time token + WebSocket handshake)

 Day 7: Deploy to AWS + First End-to-End Test
  ☐ Deploy FastAPI to AWS Lightsail (Mumbai)
  ☐ Deploy React frontend to S3 + CloudFront
  ☐ Test full flow: login → pair connector → sync data → verify in DB
  EXIT CRITERION: Signed-in user can see synced Tally company data in cloud DB

═══════════════════════════════════════════════════════════════════════════
 WEEK 2 (Days 8–14): Financial Engine + Core Reports + UI
═══════════════════════════════════════════════════════════════════════════

 Day 8–9: Deterministic Financial Engine
  ☐ Implement Python Decimal(2) financial computation module
  ☐ Receivables aging buckets (0–30, 31–60, 61–90, 90+)
  ☐ Cash & bank balance consolidator (filter by 'Cash' and 'Bank Accounts' groups)
  ☐ P&L summary (Revenue − Expenses = Net Profit)
  ☐ Period comparison (current vs previous month/year)
  ☐ WRITE UNIT TESTS: compare outputs vs. manually verified Tally reports

 Day 10–11: Report API Endpoints
  ☐ GET /reports/trial-balance
  ☐ GET /reports/profit-loss
  ☐ GET /reports/outstanding-bills (receivable + payable with aging)
  ☐ GET /reports/ledger-statement (live from Tally via connector)
```

```
  ☐ Implement data freshness metadata in all responses

 Day 12-14: Frontend Dashboard
  ☐ Login page + Google OAuth flow
  ☐ Company selector + sync status badge (●LIVE / ●SYNCING / ●OFFLINE)
  ☐ Dashboard: Cash card, Sales card, Top Debtors chart, Overdue summary
  ☐ Outstanding Bills page with aging bands (color-coded)
  EXIT CRITERION: End-to-end: login → view dashboard → see matching figures from
Tally

═══════════════════════════════════════════════════════════════════════════════
 WEEK 3 (Days 15-21): AI Chat + Exports + Hardening
═══════════════════════════════════════════════════════════════════════════════

 Day 15-16: AI Orchestration Layer
  ☐ Integrate Gemini 2.0 Flash via LiteLLM
  ☐ Build intent classification prompt (structured JSON output)
  ☐ Implement tool allowlist and parameter guard
  ☐ Build tool-to-function router (10 allowlisted tools)
  ☐ Source citation metadata attached to every response

 Day 17-18: Chat Interface
  ☐ Chat page with SSE streaming (text/event-stream)
  ☐ Message bubbles with source citation badges
  ☐ Suggested question chips
  ☐ Quick questions: "Show P&L", "Who owes me?", "Cash position"
  ☐ Conversation history sidebar

 Day 19-20: Exports + Rate Limiting
  ☐ XLSX export for reports (openpyxl)
  ☐ Basic PDF export (weasyprint or reportlab)
  ☐ S3 upload with pre-signed URL (15-min TTL)
  ☐ Redis-backed rate limiting (50 queries/user/hour)
  ☐ Retry logic and idempotency for sync jobs

 Day 21: Error Handling & Edge Cases
  ☐ Tally offline → graceful degradation with freshness warning
  ☐ Empty company data → empty state UI (not crash)
  ☐ Ambiguous date → clarification prompt
  ☐ Unsupported question → safe refusal message
  EXIT CRITERION: 10 sample questions return verified, source-cited answers

═══════════════════════════════════════════════════════════════════════════════
 WEEK 4 (Days 22-30): Security, Testing & Pilot Launch
═══════════════════════════════════════════════════════════════════════════════

 Day 22-23: Security Audit
  ☐ Cross-tenant access test (user A cannot see user B's data)
  ☐ JWT token rotation and expiry test
  ☐ SQL injection attempt on all string inputs
  ☐ Prompt injection test (malicious ledger names in chat)
  ☐ Audit log verification (every query logged)
```

```
Day 24-25: Connector Packaging
 ☐ Package Node.js connector as single Windows .exe (using pkg/SEA)
 ☐ Auto-update check on startup
 ☐ Test installation on a clean Windows machine (no Node pre-installed)
 ☐ One-page setup guide

Day 26-27: Pilot Customer Onboarding
 ☐ Onboard 2-3 friendly pilot customers (CAs or business owners)
 ☐ Monitor connector uptime, sync success rate, query latency
 ☐ Fix priority bugs found in real usage

Day 28-30: Polish & Documentation
 ☐ Add loading skeletons and micro-animations
 ☐ Write user onboarding guide (3-step setup)
 ☐ Write internal runbook (deployment, rollback, DB backup/restore)
 ☐ Document known limitations and v2 backlog
 EXIT CRITERION: Pilot customers can connect Tally, ask supported questions,
 receive verified answers, and report no critical data accuracy issues.
```

Milestone & Acceptance Criteria

Milestone	Day	Exit Criterion
M1: Tally Extraction PoC	Day 4	Extract 5 report types; totals match Tally
M2: Cloud Sync Working	Day 7	Login + pair + sync + verify data in DB
M3: Financial Engine Verified	Day 14	Dashboard figures = Tally figures for test company
M4: AI Chat Working	Day 21	10 sample questions answered with source citations
M5: Security Pass	Day 23	Zero cross-tenant access; audit logs complete
M6: Pilot Launch	Day 30	3+ real users using the product with no critical bugs

---

14. Known Bottlenecks & Risk Register

#	Risk	Probability	Impact	Mitigation
R1	Tally XML port blocked by antivirus or Windows Defender on customer machine	HIGH	HIGH	Test on 3+ clean Windows setups; document allow-list instructions; add diagnostic tool to connector
R2	Tally Educational Edition gives incorrect/partial data	MEDIUM	HIGH	Detect and reject educational version at connector startup with clear error message
R3	Large voucher databases (100k+ entries) cause slow initial sync	HIGH	MEDIUM	Batch sync (1000 records/request); progress indicator in connector tray; set expectation in onboarding

#	Risk	Probability	Impact	Mitigation
R4	Customer runs Tally on a server VM with no direct desktop access	MEDIUM	HIGH	Document: connector must run on same machine as Tally; cloud Tally (TSS) requires testing
R5	LLM provides confidently wrong financial commentary	MEDIUM	HIGH	Anti-hallucination prompts; never let LLM perform arithmetic; source citations make errors detectable
R6	Supabase free tier limits hit during pilot	LOW	MEDIUM	Pro plan already budgeted (₹2,100/month); upgrade trigger is clear
R7	Single developer + AI tools cannot deliver full scope in 30 days	HIGH	HIGH	**Mitigated by explicit scope reduction**: pilot excludes GST reconciliation, WhatsApp, mobile app, and write operations. These are v2.
R8	Gemini API pricing change or outage	LOW	MEDIUM	LiteLLM wrapper enables provider switch in 2 config lines; OpenAI is fallback
R9	Data residency compliance requirements from enterprise customers	LOW	HIGH	AWS Mumbai (ap-south-1) for all data; no data leaves India; document this in privacy policy
R10	Prompt injection via malicious Tally ledger names	MEDIUM	HIGH	Ledger names treated as untrusted data; never injected raw into LLM prompts; escaped before any use

The Critical Bottleneck: Tally Availability

```
SCENARIO: Business owner wants to check cash balance at 9:30 PM
PROBLEM:  Tally is usually closed after office hours
SOLUTION: E-Munshi serves last synced data with a visible timestamp badge
           "Cash balance ₹18.4L — synced 6:47 PM today — Tally currently offline"
           For P&L (requires live query): shows last cached version with explicit
warning
           Never serves stale data silently.
```

---

15. Open Questions — Must Resolve Before Development

The following questions **must be answered** before or during the first 3 days of development. Blocking answers are marked 🔴.

#	Question	Why It Matters	Owner	Priority
Q1	🔴 Which Tally version(s) do the pilot customers use? (Prime 3.x, 4.x, ERP 9?)	TDL XML query templates differ between versions; some bill-wise fields missing in older versions	Customer discovery	BLOCKING
Q2	🔴 Is Tally running on a local desktop PC, LAN server, or cloud VM (TSS) at pilot sites?	Cloud/VM Tally requires testing connector connectivity that may fail behind RDP/NAT	Technical audit	BLOCKING
Q3	🔴 What is the approximate voucher volume per company? (< 5k / 5k–50k / 50k–500k entries/year)	Affects initial sync time, batching strategy, and user expectations	Customer discovery	BLOCKING
Q4	🔴 Are pilot customers' Tally companies single-company or multi-company?	Multi-company users need company selection flow from day 1	Customer discovery	BLOCKING
Q5	🔴 What constitutes "correct" answers? Which Tally reports will be used as ground truth for validation?	Without agreed reference outputs, we cannot verify financial accuracy	Product owner	BLOCKING
Q6	Who will manage the Windows connector installation at pilot sites — the business owner or their CA?	Affects UX of installer (technical vs. non-technical user)	Customer discovery	HIGH
Q7	Should chat conversation history persist across sessions, and for how long?	Data retention + privacy policy implications	Product owner	HIGH
Q8	Does the client require a white-label option (custom domain / logo) for CA firms?	Architecture impact on domain routing and branding tokens	Business	HIGH
Q9	What is the regulatory stance on storing Tally voucher data in cloud? Any specific customer DPA requirements?	Data Processing Agreement needed before onboarding enterprise customers	Legal / Business	HIGH
Q10	Is there an existing E-Munshi frontend codebase to be integrated, or building UI from scratch?	Changes Week 1 timeline significantly	Engineering	MEDIUM
Q11	Write operations (creating journal entries) — required for any pilot customer, or strictly deferred?	Enabling writes changes the entire trust model, security review scope, and timeline	Product owner	MEDIUM

#	Question	Why It Matters	Owner	Priority
Q12	What domains/emails are available? (emunshi.in, emunshi.com, getmunshi.in)	Register before a competitor does	Business	MEDIUM
Q13	Should the connector auto-update silently or prompt the user for confirmation?	Windows auto-update policies vary; enterprise customers may require manual updates	Engineering	LOW

---

16. Pilot Release Checklist

The following criteria must pass before any real business owner or accountant connects their live Tally data.

Security (All MUST PASS)

- [ ] Cross-tenant data access test: User A confirmed unable to see User B's company data
- [ ] JWT expiry and rotation tested (both access and refresh tokens)
- [ ] All string inputs validated through Pydantic schemas; SQL injection attempted and blocked
- [ ] Prompt injection test: ledger name containing `IGNORE ALL PREVIOUS INSTRUCTIONS` processed safely
- [ ] Connector BLOCK_WRITE=1 verified: write tools return 405/disabled error
- [ ] Audit log contains entries for every report fetch and chat query
- [ ] HTTPS enforced on all endpoints (HTTP → HTTPS redirect active)

Financial Accuracy (All MUST PASS)

- [ ] Trial balance totals match Tally's own Trial Balance report for 3 test periods
- [ ] P&L net profit matches Tally's P&L for 3 test periods
- [ ] Outstanding receivables total matches Tally's Outstandings report as of 3 different dates
- [ ] Cash & bank balance matches Tally's Balance Sheet cash group as of today
- [ ] Aging bucket calculations verified manually for 10 overdue bills

Reliability (All MUST PASS)

- [ ] Connector tested on 2 different Windows machines (fresh install, no Node.js pre-installed)
- [ ] Tally restart during sync: connector reconnects automatically within 60 seconds
- [ ] Network drop during sync: resumes from last watermark without duplicate data
- [ ] Large dataset test: 50,000+ voucher company synced without Tally UI freezing
- [ ] Database backup and restore tested: recovery time < 30 minutes

Operations (All MUST PASS)

- [ ] Production monitoring active: Sentry error alerts + uptime check every 5 minutes
- [ ] AWS RDS automated backups enabled (7-day retention)
- [ ] Secrets not in source code or version control (AWS Secrets Manager / environment variables)

- [ ] Rollback procedure documented and tested

---

17. Architecture Decision Records (ADRs)

ADR-001: Python + FastAPI over Node.js for Cloud Backend

Status	Accepted
Context	tally-mcp-server uses Node.js/TypeScript; could reuse language for backend
Decision	Python + FastAPI for cloud backend; Node.js only for Windows connector
Rationale	Python's `decimal` module is essential for financial arithmetic correctness. AI/ML ecosystem (LangChain, LiteLLM, pandas) is Python-native. FastAPI's Pydantic validation is superior for financial data schemas.
Trade-offs	Two languages in the codebase; mitigated by clear package boundaries

ADR-002: Outbound-Only Connector (Zero Port Forwarding)

Status	Accepted
Context	Alternatives: expose Tally port via ngrok tunnel; expose connector as HTTP server
Decision	Connector makes outbound WebSocket connection to cloud; no inbound ports
Rationale	95%+ of Indian MSME offices have NAT routers with no static IP. ngrok is a third-party dependency risk. Outbound WS works through virtually any firewall configuration without customer IT intervention.
Trade-offs	Connector must be running for live queries; mitigated by sync cache

ADR-003: Read-Only Pilot (No Tally Write Operations)

Status	Accepted
Context	Riko AI offers write-to-Tally with approval; some may expect this from E-Munshi

Decision	BLOCK_WRITE=1 hardcoded in pilot connector; no write API endpoints in v1
Rationale	A bug in a write operation corrupts customer books — a catastrophic trust failure that ends the product. Read-only is much simpler to audit, test, and prove correct. Write operations require a full approval workflow, audit trail, and rollback mechanism.
Trade-offs	Slightly lower feature parity with Riko AI v1; offset by faster time-to-market and lower risk

ADR-004: Delta Sync vs. Live Query

Status	Accepted
Context	Alternative: query Tally live for every user request
Decision	Delta sync every 15 minutes into PostgreSQL; live queries only for ledger statements
Rationale	Live queries block Tally UI (10–30s for 50k+ voucher companies). Tally is closed after office hours. Users need <5s response times. Delta sync with freshness labels is the industry-proven pattern (Riko AI uses it too: "syncs every few minutes").
Trade-offs	Data up to 15 minutes stale; mitigated by visible freshness badge on every data display

ADR-005: Gemini 2.0 Flash as Primary LLM

Status	Accepted
Context	OpenAI GPT-4o-mini is the common default choice
Decision	Google Gemini 2.0 Flash via LiteLLM wrapper
Rationale	50% lower per-token cost vs GPT-4o-mini. 1M token context window. Strong multilingual support for Indian languages. LiteLLM wrapper ensures no vendor lock-in.
Trade-offs	Gemini is less battle-tested in production accounting apps than OpenAI; mitigated by LiteLLM fallback

ADR-006: PostgreSQL RLS as Primary Tenant Isolation

Status	Accepted
Context	Alternative: application-level tenant filtering only
Decision	PostgreSQL Row-Level Security (RLS) + application-level filtering (defense in depth)
Rationale	Financial data demands defense-in-depth. A bug in application-level filtering cannot be the only protection against cross-tenant data leaks. RLS enforced at database level is a hard guarantee.
Trade-offs	Slightly more complex migration and query design; eliminated risk of application-layer bugs exposing data

---

Document prepared by: E-Munshi Engineering Team
Classification: Confidential — Client Presentation
Version: 1.0 | October 23, 2026
Next review: After Open Questions (Section 15) are resolved